

Below is where we have performed the ingestion

1. Health Check

The screenshot shows the Postman interface with a collection named 'RAG MongoDB - Data Ingestion'. The first request, '1. Health Check', is a GET request to the endpoint '({baseUrl}) /api/health'. The response is a 200 OK status with a response time of 16 ms and a body of 314 B. The JSON response is as follows:

```
1 {
2   "status": "OK",
3   "message": "Server is running"
4 }
```

2. Upload Excel - Test Cases

The screenshot shows the Postman interface with the same collection. The second request, '2. Upload Excel - Test Cases', is a POST request to the endpoint '({baseUrl}) /api/upload-excel'. The response is a 200 OK status with a response time of 1.17 s and a body of 615 B. The JSON response is as follows:

```
1 {
2   "success": true,
3   "message": "Test cases file converted successfully",
4   "outputFile": "converted-1789498040010.json",
5   "output": "⚠️ Sheet \"Testcases\" not found. Using first sheet: \"testcases\"\\n✅ Converted 6354 rows from \\n\"testcases\" into C:/Users/india/Documents/GenAITraining/rag-mongo-demo-backend/src/data/converted-1789498040010.json\\n"
6 }
```

Additionally, a script is visible in the 'Scripts' tab, which saves the output file name for future requests:

```
1 // Save the output file name for next requests
2 if (pm.response.code === 200) {
3   var jsonData = pm.response.json();
4   if (jsonData.outputFile) {
5     pm.collectionVariables.set('outputFile', jsonData.outputFile);
6     console.log('Output file saved: ' + jsonData.outputFile);
7   }
8 }
```

4. List Available Files

The screenshot shows a Postman interface with a GET request to `[[baseUrl]] /api/files`. The response is a 200 OK status with a 6 ms response time and 692 B of data. The JSON response is as follows:

```
1 [
2   {
3     "name": "converted-1789498040010.json",
4     "path": "C:\\Users\\india\\Documents\\GenAITraining\\rag-mongo-demo-backend\\src\\data\\converted-1789498040010.json",
5     "size": 6600526,
6     "modified": "2026-09-15T18:47:21.088Z",
7     "type": "json"
8   }
9 ]
```

5. Create Embeddings - Batch (Test Cases - FAST)

The screenshot shows a POST request to `[[baseUrl]] /api/create-embeddings-batch` with a raw body. The response is a 200 OK status with a 489 ms response time and 447 B of data. The JSON response is as follows:

```
1 {
2   "success": true,
3   "jobId": "job-1789498540581-5mz18ghq8",
4   "message": "Batch embedding creation started using create-embeddings-batch-mistral.js",
5   "filesCount": 1,
6   "jobType": "testcases"
7 }
```

Verified MongoDB as well below

[testcases]

 MISTRAL AI BATCH PROCESSING COMPLETE!

 Final Statistics:

 Total Time: 2m 16s

 Processing Rate: 46.8 testcases/second

 Total Test Cases: 6354

[testcases] ☒ Successfully Processed: 6354

☒ Failed: 0

 Success Rate: 100.0%

 Total Cost: \$0.117741

[testcases]  Total Tokens: 11,77,411

 Average Cost per Test Case: \$0.00001853

 Average Tokens per Test Case: 185

 Cost Savings: ~90% cheaper than OpenAI embeddings

☒ Batch script completed successfully!

